

✓ *Labsheet 10*

```
import numpy as np
import matplotlib.pyplot as plt

import tensorflow as tf
from tensorflow.keras import layers, models

from sklearn.metrics import (
    confusion_matrix,
    classification_report
)

from google.colab import files
from PIL import Image

print("Libraries imported successfully!")
```

Libraries imported successfully!

```
# Load MNIST handwritten digit dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

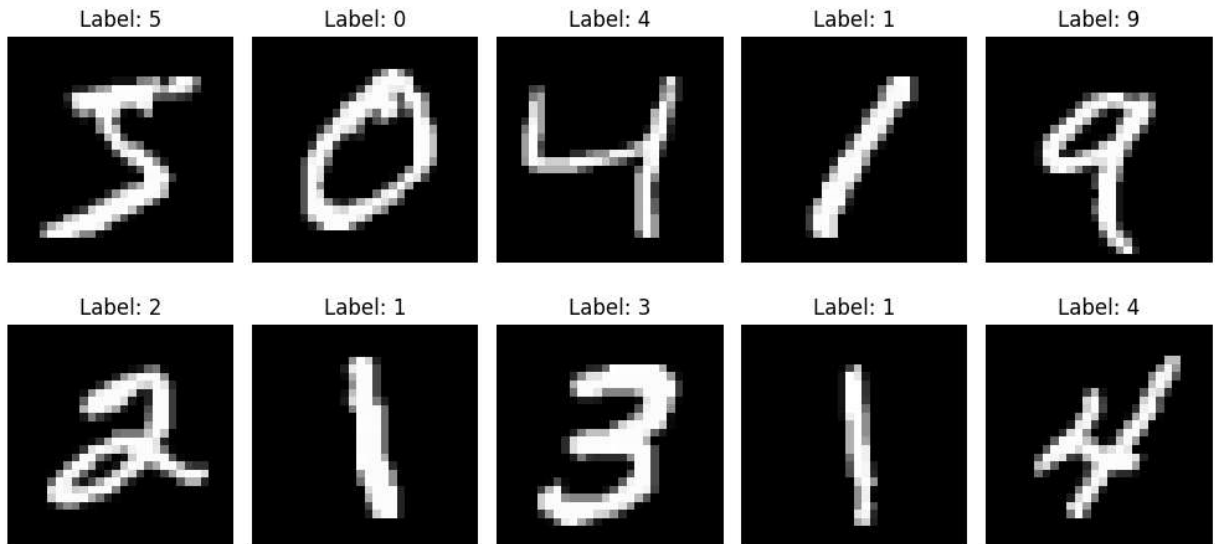
print("Training images:", x_train.shape)
print("Testing images:", x_test.shape)
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-dataset/11490434/11490434> 0s 0us/step
Training images: (60000, 28, 28)
Testing images: (10000, 28, 28)

```
plt.figure(figsize=(10, 5))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_train[i], cmap="gray")
    plt.title("Label: " + str(y_train[i]))
    plt.axis("off")

plt.tight_layout()
plt.show()
```



```
# Normalize pixel values from 0-255 to 0-1
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
```

```
# Add channel dimension
x_train = np.expand_dims(x_train, axis=-1)
x_test = np.expand_dims(x_test, axis=-1)
```

```
print("Training shape:", x_train.shape)
print("Testing shape:", x_test.shape)
```

```
Training shape: (60000, 28, 28, 1)
Testing shape: (10000, 28, 28, 1)
```

```
model = models.Sequential([

    # Convolution Layer
    layers.Conv2D(
        32,
        (3, 3),
        activation="relu",
        input_shape=(28, 28, 1)
    ),

    # Pooling Layer
    layers.MaxPooling2D((2, 2)),

    # Second Convolution Layer
    layers.Conv2D(
        64,
        (3, 3),
        activation="relu"
    ),

    # Second Pooling Layer
```

```

layers.MaxPooling2D((2, 2)),

# Convert feature maps into 1D
layers.Flatten(),

# Fully Connected Layer
layers.Dense(
    128,
    activation="relu"
),

# Output Layer
layers.Dense(
    10,
    activation="softmax"
)
])

model.summary()

```

```

/usr/local/lib/python3.13/dist-packages/keras/src/layers/convolutional/base_conv
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dense_1 (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)
Trainable params: 225,034 (879.04 KB)
Non-trainable params: 0 (0.00 B)

```

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

print("Model compiled successfully!")

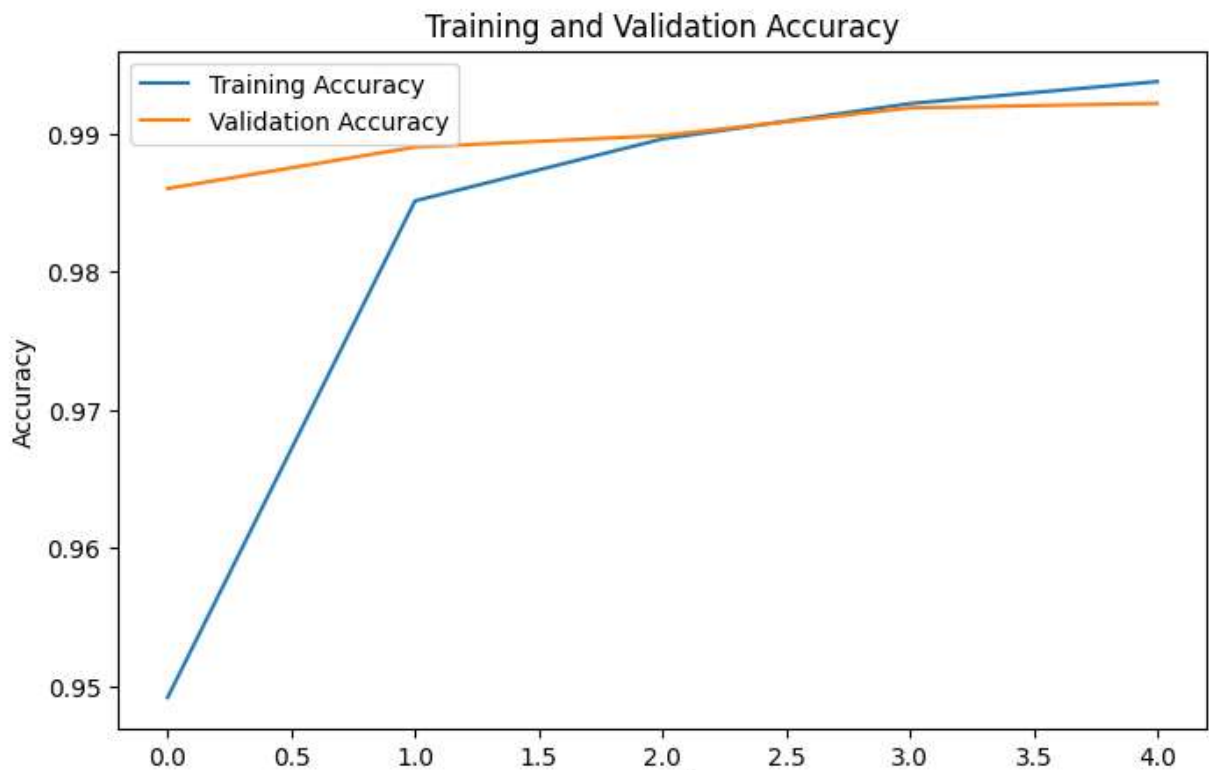
```

Model compiled successfully!

```
history = model.fit(  
    x_train,  
    y_train,  
    epochs=5,  
    batch_size=64,  
    validation_split=0.1  
)
```

```
Epoch 1/5  
844/844 ————— 67s 78ms/step - accuracy: 0.9492 - loss: 0.1710 - v  
Epoch 2/5  
844/844 ————— 62s 54ms/step - accuracy: 0.9851 - loss: 0.0500 - v  
Epoch 3/5  
844/844 ————— 45s 53ms/step - accuracy: 0.9896 - loss: 0.0332 - v  
Epoch 4/5  
844/844 ————— 47s 55ms/step - accuracy: 0.9921 - loss: 0.0244 - v  
Epoch 5/5  
844/844 ————— 80s 53ms/step - accuracy: 0.9937 - loss: 0.0191 - v
```

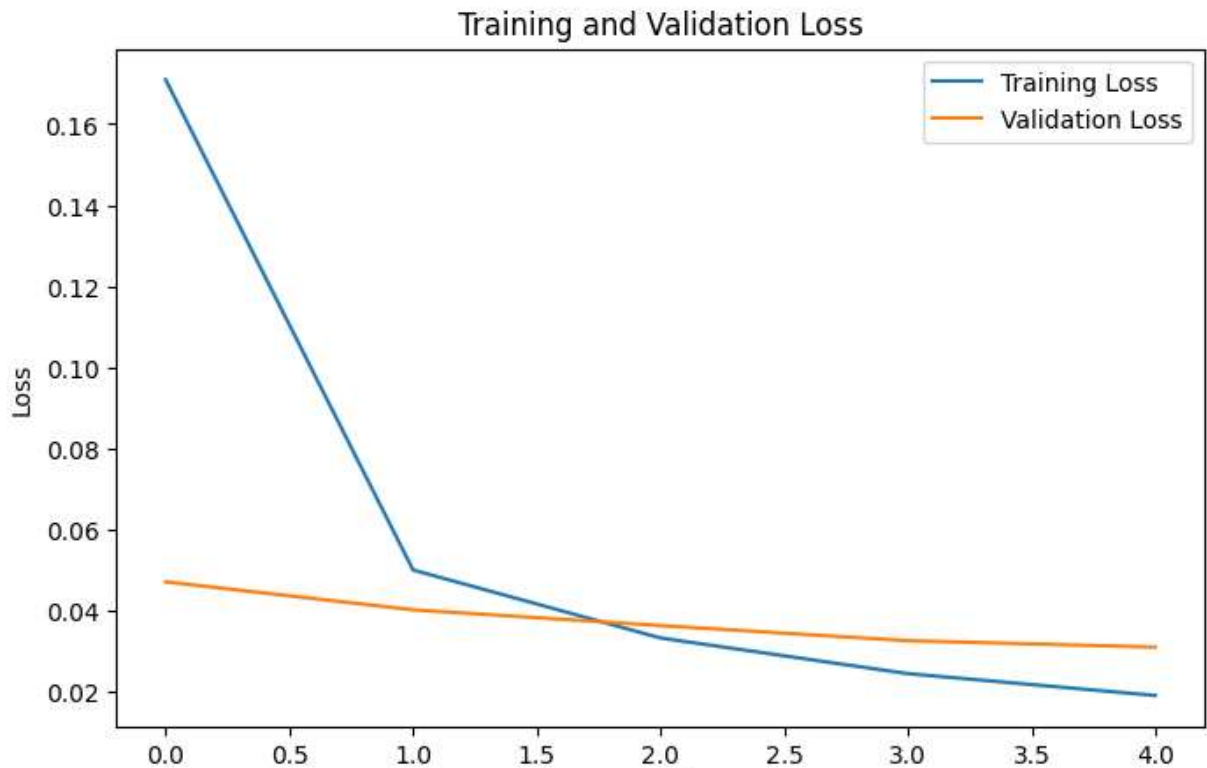
```
plt.figure(figsize=(8, 5))  
  
plt.plot(history.history["accuracy"], label="Training Accuracy")  
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")  
  
plt.title("Training and Validation Accuracy")  
plt.xlabel("Epoch")  
plt.ylabel("Accuracy")  
plt.legend()  
plt.show()
```



```
plt.figure(figsize=(8, 5))

plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")

plt.title("Training and Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()
```



```
test_loss, test_accuracy = model.evaluate(
    x_test,
    y_test,
    verbose=0
)
```

```
print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)
```

```
Test Loss: 0.03376936540007591
Test Accuracy: 0.9887999892234802
```

```
# Predict test images
predictions = model.predict(x_test)

# Convert probabilities to class labels
y_pred = np.argmax(predictions, axis=1)

print("Predictions generated successfully!")
```

```
313/313 ————— 3s 11ms/step
Predictions generated successfully!
```

```
print(classification_report(
    y_test,
    y_pred
))
```

	precision	recall	f1-score	support
0	0.99	1.00	0.99	980
1	0.99	1.00	0.99	1135
2	0.99	0.98	0.99	1032
3	0.99	0.99	0.99	1010
4	0.99	0.99	0.99	982
5	0.98	0.99	0.98	892
6	0.99	0.99	0.99	958
7	0.99	0.99	0.99	1028
8	0.98	0.99	0.98	974
9	0.99	0.97	0.98	1009
accuracy			0.99	10000
macro avg	0.99	0.99	0.99	10000
weighted avg	0.99	0.99	0.99	10000

```

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(8, 6))

plt.imshow(cm)

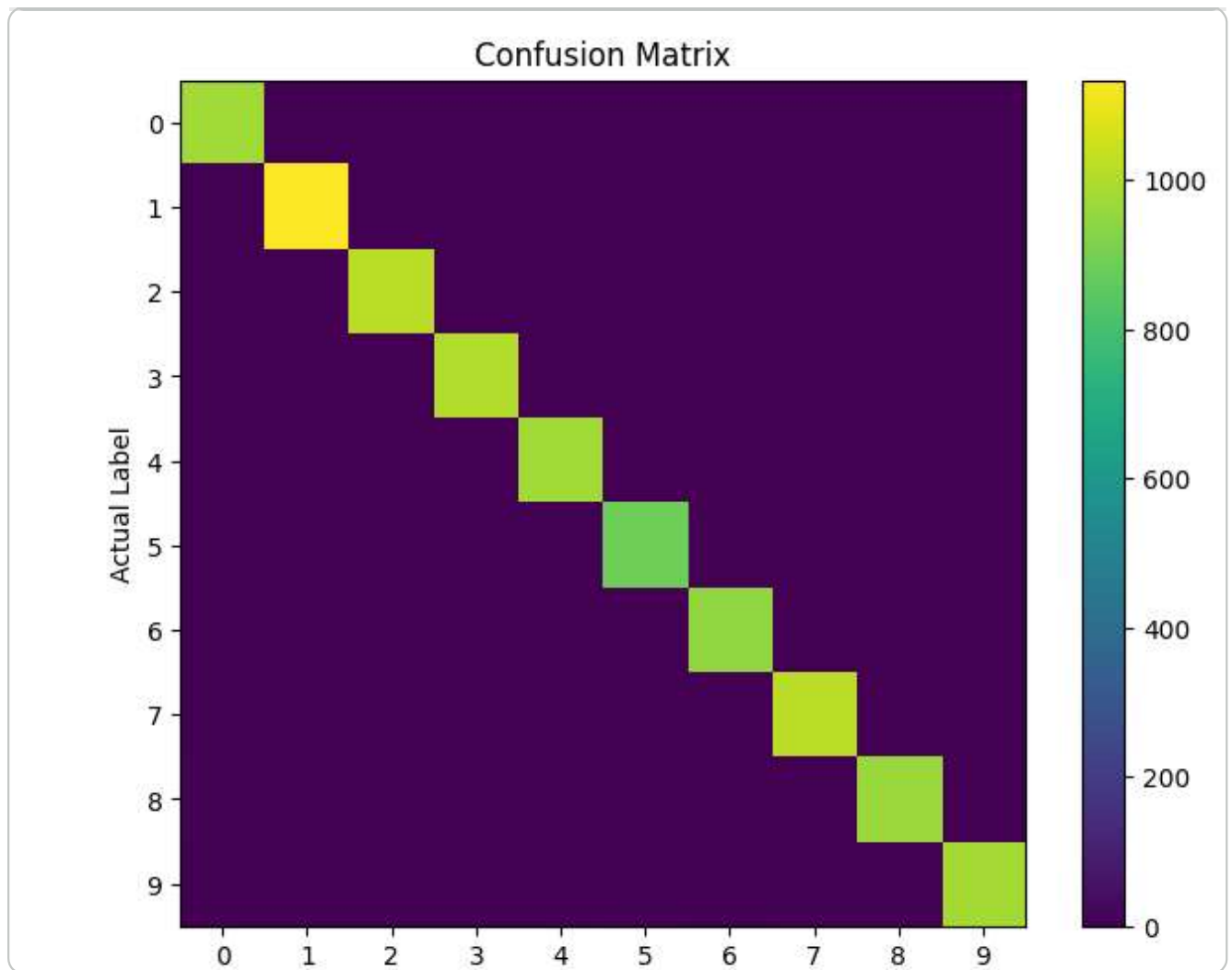
plt.title("Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")

plt.colorbar()

plt.xticks(range(10))
plt.yticks(range(10))

plt.show()

```



QUESTIONS & ANSWERS Q1. What is image classification? How does it differ from object detection and image segmentation?

Answer:

Image classification is the process of assigning a class or category to an image.

For example, a CNN can classify an image as 0, 1, 2, 3, etc.

Method Meaning Image Classification Assigns a class to an image Object Detection

Identifies objects and their locations using bounding boxes Image Segmentation

Classifies individual pixels into different regions

The practical specifically asks for this comparison.

Q2. Explain the architecture and working principle of CNN.

Answer:

A CNN is a deep learning architecture used mainly for image processing.

Its main components are:

Convolution Layer – extracts important features from the image. Activation Function – introduces non-linearity. Pooling Layer – reduces the size of feature maps. Flatten Layer – converts feature maps into a one-dimensional vector. Fully Connected Layer – performs classification. Output Layer – gives the final class probabilities.

The CNN learns important image features automatically during training.

Q3. What is the role of Convolution, Pooling and Fully Connected Layers?

Answer:

Convolution Layer

It extracts features such as edges, shapes and patterns from an image.

Pooling Layer

It reduces the spatial size of feature maps and helps reduce computational complexity.

Fully Connected Layer

It uses the extracted features to perform the final classification.

Q4. Why is image normalization performed?

Answer:

Image normalization converts pixel values into a smaller and consistent range.

For example:

Original pixel range = 0 to 255 Normalized range = 0 to 1

Normalization helps the neural network train more efficiently and provides stable learning.

The experiment specifically includes normalization as part of preprocessing.

Q5. Explain ReLU and Softmax.

Answer:

ReLU

ReLU stands for Rectified Linear Unit.

It is commonly used in hidden layers.

$\text{ReLU}(x) = \max(0, x)$

It introduces non-linearity into the CNN.

Softmax

Softmax is generally used in the output layer for multi-class classification.

It converts the output values into probabilities for different classes.

In our code:

```
layers.Dense(10, activation="softmax")
```

There are 10 output classes because MNIST contains digits 0–9.

Q6. What is a Confusion Matrix?

Answer:

A confusion matrix is a table used to compare actual labels with predicted labels.

For a digit classification problem, it shows how many images of each digit were correctly or incorrectly classified.

For example:

Actual 5 → Predicted 5 = Correct Actual 5 → Predicted 3 = Incorrect

It helps identify which classes are being confused by the model.

Q7. Differentiate Accuracy, Precision, Recall and F1-Score.

Answer:

Metric Meaning Accuracy Overall percentage of correct predictions Precision How many predicted positive samples are actually positive Recall How many actual positive samples are correctly identified F1-Score Harmonic mean of Precision and Recall Formulas

Accuracy:

Correct Predictions / Total Predictions

Precision:

$TP / (TP + FP)$

Recall:

$TP / (TP + FN)$

F1-Score:

$2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$